

Performance Testing

Practica con k6 sobre TaskFlow API

#	Titulo	Tiempo	Objetivo
0	Setup y verificacion del entorno	10 min	API corriendo + k6 instalado
1	Smoke test — diagnosticar el script inicial	20 min	Script corregido, corriendo verde
2	Load test con thresholds (SLOs)	25 min	Reporte con p95 y error_rate
3	Scenario spike — interpretar resultados	15 min	Analisis escrito del reporte
4	Diagnosticar el bug de performance	15 min	Sintoma + causa probable
—	Cierre y conexion con CI/CD	5 min	—

Trabajo en equipos. Cada equipo entrega el script k6 corregido y el analisis escrito (secciones marcadas con lineas de respuesta).

P A R T E 0 10 min

Setup y verificacion del entorno

Objetivo

Confirmar que la API de TaskFlow corre correctamente y que k6 esta instalado antes de arrancar el ejercicio principal.

Pasos

1. **Levantar la API de TaskFlow.** Desde la raiz del repo:

```
bash
npm run dev
# La API debe escuchar en http://localhost:3001
```

2. **Verificar que el seed tiene datos.** Con curl o Thunder Client:

```
bash
```

```
# Autenticarse con el usuario de seed
curl -s -X POST http://localhost:3001/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"alice@taskflow.dev","password":"Password1"}' | jq

# Si responde con token: el seed esta ok
```

3. **Verificar k6.** En una terminal nueva:

```
bash
k6 version
# Debe mostrar v0.50.x o superior
```

Si k6 no esta instalado

macOS: `brew install k6`

Linux: `sudo snap install k6`

Windows: `winget install k6` o descargar desde k6.io/docs/getting-started/installation

4. **Revisar el script inicial.** Abrir `performance/scenarios/api-load.k6.js` y leerlo completo (2 minutos). No lo editen todavia.

P A R T E 1 20 min

Smoke test — diagnosticar el script inicial

Objetivo

Correr el script tal como esta en el repo, leer los errores, identificar los bugs y corregirlos hasta que el smoke test pase en verde.

1.1 — Correr el script tal como esta (sin modificarlo)

```
bash
k6 run --env BASE_URL=http://localhost:3001 \
  --vus 1 --duration 10s \
  performance/scenarios/api-load.k6.js
```

Por que --vus 1 --duration 10s?

Anulamos los escenarios definidos en options y corremos con 1 solo VU durante 10 segundos.

Esto es un smoke test: queremos saber si el script funciona en absoluto, antes de agregar carga.

1.2 — Registrar lo que pasa

Observen el output de k6. Anoten los errores y comportamientos inesperados:

ERRORES / CHECKS FALLIDOS QUE OBSERVAN:

Al correr el script original, se observan 3 fallos:

login status 200: 0% de éxito (Falló en todas las iteraciones).

error_rate: 50% (Debido a fallos en login y rutas incorrectas).

tasks_list_duration: 0ms (La sección de tareas nunca se ejecutó porque dependía de un login exitoso y proyectos existentes).

1.3 — Identificar los bugs del script

El script tiene tres problemas que hacen que los checks fallen o que parte del test nunca se ejecute. Usando el output anterior y leyendo el código, completar:

Descripcion del problema	Linea(s) afectada(s) en el script
Bug 1 — URL: Falta el prefijo <code>/api</code> en los endpoints de registro, login, proyectos y tareas.	55, 72, 84, 93
Bug 2 — Login: El script utiliza credenciales hardcodeadas para el login, ignorando el token del setup.	71-79
Bug 3 — Datos: El GET de proyectos devuelve una lista vacía y el script intenta acceder a un ID inexistente.	54-62 (setup incompleto), 91 (error de asignacion)

Pista si no encuentran el Bug 3

El setup() registra un usuario nuevo con email `perf-{timestamp}@test.com`.

Ese usuario no tiene proyectos. Entonces, ¿qué devuelve `GET /api/projects`?

Y si `projectId` es `undefined`, ¿qué pasa con el paso de tareas?

1.4 — Corregir el script

Aplicar las tres correcciones. El script corregido debe:

- Usar el prefijo correcto en todas las URLs (revisar Clase 4 para confirmar la base de los endpoints).
- Que el VU use el token creado en `setup()`, en lugar de hacer un login separado con credenciales hardcodeadas.
- Que el `setup()` cree un proyecto y una tarea para el usuario que registra, de modo que `GET /api/projects` devuelva datos reales.

Estructura del setup() corregido

El `setup()` debe: (1) registrar usuario, (2) crear proyecto con `POST /api/projects`, (3) crear una tarea con `POST /api/projects/:id/tasks`, (4) retornar `{ token, projectId }`.

El default function recibe ese objeto y lo usa en los headers y en las URLs.

1.5 — Verificar que el smoke test pasa

```
bash
k6 run --env BASE_URL=http://localhost:3001 \
      --vus 1 --duration 15s \
      performance/scenarios/api-load.k6.js
```

El resultado esperado:

- login status 200: 100%
- projects status 200: 100%
- tasks status 200: 100%
- error_rate: 0.00%

Cuando el smoke test pasa en verde, avancen a la Parte 2.

Scaffold del script corregido

Pueden usar este scaffold como punto de partida. Los comentarios TODO indican lo que deben completar:

```
javascript
import http from 'k6/http'
import { check, sleep } from 'k6'
import { Rate, Trend } from 'k6/metrics'

const errorRate = new Rate('error_rate')
const listDuration = new Trend('list_duration', true)
const tasksDuration = new Trend('tasks_duration', true)

export const options = {
  thresholds: {
    http_req_duration: ['p(95)<500', 'p(99)<1000'],
    error_rate: ['rate<0.01'],
    list_duration: ['p(95)<400'],
    tasks_duration: ['p(95)<400'],
  },
  scenarios: {
    load: {
      executor: 'ramping-vus',
      startVUs: 0,
      stages: [
        { duration: '30s', target: 50 },
        { duration: '1m', target: 50 },
        { duration: '30s', target: 0 },
      ],
    },
    spike: {
      executor: 'ramping-vus',
      startVUs: 0,
      stages: [
        { duration: '10s', target: 200 },
        { duration: '30s', target: 200 },
        { duration: '10s', target: 0 },
      ],
    },
  },
}
```

```

    startTime: '3m',
  },
},
}

const BASE_URL = __ENV.BASE_URL || 'http://localhost:3001'

export function setup() {
  // TODO 1: Registrar usuario con email dinamico
  // POST /api/auth/register { email, password, name }
  // Guardar el token de la respuesta

  // TODO 2: Crear un proyecto para ese usuario
  // POST /api/projects { name, description }
  // Necesitan pasar el token en Authorization: Bearer <token>
  // Guardar el projectId de la respuesta

  // TODO 3: Crear una tarea dentro del proyecto
  // POST /api/projects/:projectId/tasks { title, priority }

  // TODO 4: Retornar los datos que necesita el VU
  return { token: '???', projectId: '???' }
}

export default function (data) {
  const headers = {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${data.token}`,
  }

  // Paso 1: listar proyectos del usuario
  // TODO 5: GET /api/projects con los headers correctos
  // Registrar duration en listDuration
  // Registrar en errorRate si status != 200
  // check: 'projects status 200'

  sleep(0.5)

  // Paso 2: listar tareas del proyecto creado en setup
  // TODO 6: GET /api/projects/${data.projectId}/tasks?status=TODO
  // Registrar duration en tasksDuration
  // check: 'tasks status 200'

  sleep(1)
}

```

P A R T E 2 25 min

Load test con thresholds (SLOs)

Objetivo

Correr el escenario load completo (50 VUs durante 2 minutos) y analizar si los SLOs definidos en los thresholds se cumplen.

2.1 — Correr el load test

```
bash
k6 run --env BASE_URL=http://localhost:3001 \
  --scenario load \
  performance/scenarios/api-load.k6.js
```

El flag --scenario load

Ejecuta solo el escenario 'load' (50 VUs, 2 min), ignorando el spike.

El test completo tarda ~3.5 minutos. El load solo tarda ~2 minutos.

2.2 — Registrar las metricas del reporte

Al terminar el test, k6 muestra un summary. Completar la tabla con los valores obtenidos:

Metrica	Valor obtenido	Threshold (SLO)	Cumple?
http_req_duration p95	2.99 ms	< 500 ms	<i>SI / NO</i>
http_req_duration p99	4.29 ms	< 1000 ms	<i>SI / NO</i>
error_rate	0.00%	< 1%	<i>SI / NO</i>
list_duration p95	2.76 ms	< 400 ms	<i>SI / NO</i>
tasks_duration p95	3.17 ms	< 400 ms	<i>SI / NO</i>
throughput (http_reqs/s)	50.21 reqs/s	—	—

2.3 — Analizar el reporte

Responder en base a los numeros que obtuvieron:

1. En que orden leyeron las metricas (error rate -> percentiles -> throughput -> recursos)? Que concluyen primero?

Concluimos que el sistema es estable con la carga utilizada, gracias a que el error rate es menor a 1%. Si hubieran errores, las metricas no serian confiables.

2. Si algun threshold fallo (linea roja en k6), describir el sintoma con precision: que metrica, que valor, cuanto supera al SLO.

No fallo ningun treshold. Todos los valores se acoplan a los SLO establecidos.

3. El throughput se aplano antes de los 50 VUs, o siguio creciendo linealmente? Que implica eso para el sistema?

El throughput de 50.21 reqs/s, sumado a las latencias bajas y constantes, da a entender que no se aplano antes de llegar al objetivo. Esto quiere decir que el sistema no llego a un punto de carga que pudiera causar un bottleneck y puede manejar 50 usuarios simultaneos de buena manera, e incluso deberia escalar bien con mayor carga.

PARTE 3 15 min

Scenario spike — interpretar resultados

Objetivo

Correr el scenario spike (200 VUs en pico subito) y comparar el comportamiento del sistema frente al load test.

3.1 — Correr el spike test

```
bash
k6 run --env BASE_URL=http://localhost:3001 \
  --scenario spike \
  performance/scenarios/api-load.k6.js
```

3.2 — Comparar con el load test

Completar la tabla comparativa:

Metrica	Load (50 VUs)	Spike (200 VUs)
p95 total	2.99 ms	7.18 ms
p99 total	4.29 ms	10.55 ms
error_rate	0.00%	0.00%
tasks_duration p95	3.17 ms	7.75 ms

3.3 — Preguntas de analisis

1. Usando la clasificacion del modulo teorico, a que modo de falla corresponde el comportamiento del sistema bajo spike? (Slow under all load / **Slow under heavy load** / Degrada en el tiempo / Responde pero entrega mal)

La latencia aumento a mas del doble, aunque los tiempos siguen cumpliendo de buena forma los SLOs, es probable que el sistema se haga mas lento a medida que aumenten los usuarios.

2. El sistema se recupero cuando la carga volvio a 0? Como lo pueden verificar en el reporte de k6?

Si el sistema se recupero totalmente.

El error rate se mantuvo en 0% en toda la prueba, osea que no se dieron deadlocks despues del pico de carga.

El tiempo maximo registrado (214ms) muestra que incluso a maxima presion, el sistema no deberia llegar a un estado de timeout.

3. Si tuvieran que agregarle un threshold especifico al escenario spike, que umbral eligirian para p95? Justificar: no tienen que usar el mismo que en el load test.

Dado que en este test obtuvimos 7ms, se podria bajar el treshold de p95 a < 200 ms, que iria acorde al resultado maximo obtenido y serviria como una alerta temprana en caso de que el sistema empiece a sufrir. Ademias de que demuestra una confianza en el sistema para poder cumplir con este umbral sin problema.

P A R T E 4 15 min

Diagnosticar el bug de performance

Objetivo

El endpoint GET /api/projects se degrada de forma notoria bajo el escenario spike. Usando los numeros del reporte y el codigo fuente, identificar la causa raiz.

4.1 — Observacion del sintoma

Pregunta: Comparando list_duration p95 en load vs spike, el deterioro es mayor al esperado por la diferencia de VUs? Cuanto (en ms y en %).

Si el deterioro es mayor al esperado. El incremento es de 3.82ms, que equivale a un 138% en la latencia del p95 comparado al test de carga inicial.

4.2 — Buscar la causa en el codigo

Abrir el archivo `apps/api/src/services/project.service.ts` y buscar la funcion `listProjects` (cerca de las lineas 45-55).

Responder:

1. Que condicion de filtro falta en la query de Prisma?

Falta la condicion `archived: false`

2. Que impacto tiene esa omision cuando hay muchos proyectos archivados en la base de datos?

La API recupera todos los proyectos del usuario, tanto activos como archivados, lo que aumenta de forma innecesaria el volumen de datos transferidos y el tiempo de respuesta del endpoint.

3. Usando la tabla de diagnostico de la teoria (sintoma -> causa -> donde mirar), completar para este caso:

Campo	Su respuesta
Sintoma observable	Incremento desproporcionado del p95 en la lista de proyectos bajo carga.
Causa probable	Recuperacion de registros innecesarios (archivados) por la falta de filtrado.
Donde mirarian para confirmar	En el archivo: <code>apps/api/src/services/project.service.ts</code> , linea 48 (funcion <code>listProjects</code>).
Tipo de falla (segun ISTQB §1.5)	Eficiencia en el desempeño (Performance Efficiency).

4.3 — Proponer la correccion

Escribir la linea de codigo que corrige el bug en `project.service.ts`:

```
typescript
// Agregar dentro del bloque where: { ... }
// de la funcion listProjects:

// TODO: escribir aqui la condicion que falta
```

Justificacion de la correccion (una oracion):
`archived: false`

Al filtrar por `'archived: false'`, nos aseguramos de devolver solamente los proyectos activos, reduciendo el tamaño de la respuesta y el tiempo de procesamiento en el servidor y en la base de datos.

Cierre — Conexion con CI/CD

El workflow nightly.yml del repo ya define un job que corre k6 en GitHub Actions. La logica es:

- **En cada PR:** smoke test (1 VU, 30 s). Bloquea el merge si alguna check falla.
- **En merge a main:** load test completo contra la API levantada en el job. Compara con baseline.
- **Nightly:** spike + load. Resultados en el artifact del run, no bloquean merges.

Para Hito 6 (Clase 13)

El entregable exige documentar $p95 < 500$ ms para el endpoint `POST /api/projects/:id/tasks`.

Reutilicen el script corregido de hoy como base. Solo necesitan agregar el escenario y los thresholds especificos para ese endpoint.

Entregable de la clase

Al finalizar el modulo, el repositorio de la pareja debe tener:

- `performance/scenarios/api-load.k6.js` con los tres bugs corregidos y el `setup()` funcionando.
- Este documento completado con las respuestas escritas en las secciones marcadas.
- Un archivo `performance/resultados-clase9.md` con una tabla de los valores obtenidos en el load test y el spike test.

Criterio de aprobacion de la practica

El smoke test del script corregido debe pasar con 0% error rate.

Los tres bugs identificados en 1.3 deben estar descriptos correctamente.

Las preguntas de analisis de la Parte 3 y Parte 4 deben estar respondidas.

Anexo — Referencia rapida de k6

Leer el summary de k6

k6 muestra un summary al terminar. Las secciones importantes:

```
scenarios: (100.00%) 1 scenario, 50 max VUs

  checks.....: 100.00% 240 out of 240
  data_received.....: 180 kB 1.5 kB/s
  http_req_duration.....: avg=142ms min=80ms med=130ms max=890ms
p(90)=220ms p(95)=310ms
  { expected_response:true }...: avg=142ms ...
  http_req_failed.....: 0.00% 0 out of 240
  http_reqs.....: 240 2.0/s

thresholds
  http_req_duration.....: p(95)<500 OK
  error_rate.....: rate<0.01 OK
```

Thresholds — valores de referencia

```
javascript
export const options = {
  thresholds: {
    // Todos los requests
    'http_req_duration': ['p(95)<500'],

    // Solo los requests con tag especifico
    'http_req_duration{endpoint:tasks}': ['p(95)<400'],

    // Custom metric tipo Rate
    'error_rate': ['rate<0.01'], // menos del 1%

    // Custom metric tipo Trend
    'tasks_duration': ['p(95)<400'],
  }
}
```

Tags para separar metricas por endpoint

```
javascript
// Agregar tags al request para filtrar en el reporte
const res = http.get(url, {
  headers,
  tags: { endpoint: 'tasks' }, // aparece en el summary filtrado
})

// Threshold especifico para ese tag:
```

```
// 'http_req_duration{endpoint:tasks}': ['p(95)<400']
```

Lifecycle functions de k6

```
javascript
// Orden de ejecucion:
// 1. setup()           - una vez, antes de todo
// 2. default function - una vez por VU por iteracion
// 3. teardown(data)    - una vez, al final (recibe lo que devuelve setup)

export function setup() {
  return { token: '...', projectId: '...' } // disponible en default y teardown
}

export default function(data) {
  // data.token, data.projectId
}

export function teardown(data) {
  // limpiar recursos si hace falta (opcional)
}
```